

English Comic / Webtoon OCR

Accuracy & Garble Fix — Complete Guide

CPU-only · 4 vCPU / 15 GB RAM · No GPU · Offline after download

For automated English-scanlation manhwa/webtoon → recap video pipelines
Compiled 2026-09-10 · Exhaustive research (GitHub, HF, arXiv, scanlation tools, papers)

0. Target Pipeline & Constraints

Pipeline: scrape English scanlations → panel slice → ogkalu bubble/text detect → OCR English dialogue → layered dictionary post-correction → Kokoro-82M TTS → recap video. Hardware: Azure VM, Intel Xeon 8272CL (4 vCPU), 15 GB RAM, ~55 GB disk, **no GPU**. Models must stay under ~600 M params (GOT-OCR2 ceiling). Prefer ONNX / quantized / <300 MB specialists.

Problems A–I: A=spiky ALL-CAPS font systematic misreads + token reorder; B=word-order scrambles; C=2–5% of ~21k lines garbled; D=credit/watermark bleed; E=keep SFX verbatim; F=preserve hyphenation; G=GOT too slow for all panels; H=junk word lists; I=protect proper nouns/murim terms.

1. Why Garbles Happen (Root Causes)

- **Full-page / loose crops** — art, gradients, borders read as characters (highest impact).
- **Generic detectors** — miss irregular bubbles, free text, extreme webtoon aspect ratios.
- **Wrong language pack** — Chinese RapidOCR models on English drop spaces (CER ~0.136 vs ~0.050 with EN pack).
- **Stylized ALL-CAPS fonts** — systematic substitutions + reorder (IS THIS → SI THIS / SIH OS / ESIHI).
- **No seq2seq corrector** — character tools (SymSpell, patterns) cannot fix word-order.
- **No SFX / name guards** — correctors invent English words from KEUK / romanized names.

2. Current Stack vs Accuracy-First Target

Stage	Typical current	Target	Solves
Detect	ogkalu RT-DETR + YOLOv8m	Keep + tighter crops + reject tiny boxes	A, C, D
OCR primary	RapidOCR (risk: ch pack)	RapidOCR EN pack only on crops	A, spaces
OCR secondary	Paddle v4 / none	Baberu ONNX (115 M bubble EN)	A
Hard fallback	GOT-OCR2 (~10 s/panel)	Gate strictly; rare only	G
Post-correct	Dicts + wordfreq + wordninja	+ ByT5-small on YOUR pairs + SCOWL	A, B, H, I
Guards	Partial	SFX / hyphen / name allow-lists	E, F, I

3. English Bubble / Stylized Lettering OCR Engines

Tool	Size	CPU	Problems	Effort
Baberu OCR (ONNX)	121–242 MB	Excellent	A (bubble EN)	Drop-in
RapidOCR EN pack	~15–50 MB	Already in stack	A (spaces)	Config
COMICS Text+ models	Varies	Feasible if ONNX	A (Western comic)	Moderate
MIT 48px CTC	~65 M	Good	Ensemble	Moderate
PaddleOCR-VL-For-Manga	~0.5–0.9 B	Borderline	Hard pages only	Moderate
GOT-OCR2 (you have)	~580 M	Slow ~10 s	G — gate only	Done

No public model is trained specifically on modern English webtoon spiky ALL-CAPS fonts. Closest: Baberu (bubble EN) + COMICS Text+ (Western). Your raw→corrected pairs are the best training asset.

4. Post-OCR Correction (Including Word-Order)

Character-level tools (SymSpell, Zentrix patterns, edit-distance) **cannot** fix token reorder. Need sequence models trained on your error distribution.

Tool	Size	Fixes	CPU	Notes
ByT5-small fine-tuned on YOUR pairs	~300 M	A + B (reorder)	Fits budget	Highest ROI residual fix
ByT5-base / BART / mBART	~580 M+	A + B	Heavy	Literature strong; big for 15 GB
ZentrixLabs patterns (~865)	Tiny	A typos only	Instant	No reorder
Your layered JSON dicts	Tiny	A, E, F, I	Done	Keep as first pass

Papers (ByT5 post-OCR English/Swedish/Dutch, BART WNUT 2021): seq2seq cuts CER 20–40%+ when trained on parallel noisy→clean lines. Synthetic nlpaug alone is insufficient for comic font scrambles.

5. Fine-Tune Plan: ByT5-small on Your Pairs

Target: **google/byt5-small** (~300 M). Input = OCR line → target = corrected line (optional prefix correct:). Data: start 2–5 k high-quality pairs from your 328 chapters; include reorder examples (SI THIS, CTNOHS, ...). Train: one-off Colab T4 / rented GPU, bf16, 3–10 epochs, best by val CER/WER. Infer: only on lines flagged garbled (same gate as GOT). ONNX or transformers CPU; aim <1–2 s/line. Guards: never rewrite tokens matching SFX list, hyphen splits, or series name list.

This is the path that can drive 2–5% garble → near-zero offline and kill cloud-LLM dependence for residual errors.

6. Synthetic Data for English Comic Lettering

manga-ocr synthetic generator (HTML/Chrome bubbles + fonts); docTR-Synth-Generator (FrequencyWords + degradations); StyleText research pipelines. Recipe: render your series dialogue in spiky ALL-CAPS fonts into bubble shapes → mild blur/JPEG → RapidOCR → pair with GT → mix with real pairs for ByT5 / optional recognizer FT.

7. Detectors & Reading Order

Keep ogkalu RT-DETR + YOLOv8m (among best public). Add Kiuyha YOLO26-Nano, mayocream comic-text-detector ONNX, or WORD-pytorch as ensemble. Tighter crops + area/confidence rejection still beat recognizer swaps for garble reduction. Reading order: Magi/Magiv3 (research, heavier); manga109 panel-order-estimator (geometric); within-bubble: sort line boxes top→bottom. Inside-bubble scrambles are better fixed by seq2seq corrector than global order models.

8. Dictionaries, SFX, Names (Problems E, F, H, I)

Validity: SCOWL / Hunspell en_US size 60 (or 70 large) instead of polluted wordninja — wordlist.aspell.net / en-wl GitHub. Frequency: FrequencyWords / gwordlist (filter junk). SFX: koreanling.com webtoon onomatopoeia; noveltranslator Korean action SFX; Gourmet Scans SFX notebook — keep KEUK, EUARGHH, KEUACK, KUGH, ■■■, ■, ■... verbatim. Series JSON: character names + murim/wuxia terms — never 'correct'. Preserve INSTRU- TOR style hyphenation.

9. Benchmarks & Datasets (English Comic OCR)

COMICS Text+ (arXiv 2212.14674, gsoykan/comics_text_plus) — best public English Western comic detection + recognition + cleaned text. eBDtheque (small, high quality); DCM / Comics100 / CoMix / PopManga (detection + some English). Eval offline on COMICS Text+ Recognition; always re-measure on a held-out set of **your** chapters.

10. Scanlation Community Tools (Lift What Helps)

manga-image-translator / BallonsTranslator: 48px CTC, detectors, modular. Koharu / XianScan: CPU-first, multi-OCR (Baberu/Hayai listed). ComicTL: browser YOLO26 + PaddleOCR ONNX tight bubbles. COMICS Text+ rule post-process (first/last letter + single-char). None fully solve English spiky-font reorder — your pairs remain unique.

11. Ranked Recommendations for This Server

1. Highest ROI this week

- Force RapidOCR **English pack only**; clear any ch model paths.
- Tighten ogkalu crops; reject tiny/noisy boxes.
- Add Baberu ONNX as fast secondary before GOT.
- Gate GOT strictly (only when RapidOCR + Baberu both fail).

2. Highest ROI for residual A+B (kill cloud dependence)

- Fine-tune ByT5-small on 2–5k of your raw→corrected pairs (include reorder cases).
- Deploy gated after OCR; SFX / hyphen / name allow-lists.
- Measure CER/WER + reorder cases on held-out 500 lines.

3. Dictionary hygiene

- SCOWL en_US-60 + series lists; explicit SFX and name blacklists for any correction step.

4. Avoid

- Running GOT or any ~0.5 B+ model on every panel.
- Relying only on SymSpell / synthetic-typo ByT5 for reorder.
- Expanding cloud LLM use — invest in the local corrector instead.

12. Single Highest-ROI Next Step (Week 1)

1. Export 2 000–5 000 best raw→corrected line pairs (include SI THIS / CTNOHS style errors).
2. Fine-tune google/byt5-small on Colab T4 (bf16, few epochs, best by val CER).
3. Evaluate on held-out 500 lines + reorder subset.
4. Deploy as gated post-processor after RapidOCR (+ optional Baberu) with SFX/name/hyphen guards.
5. Re-measure how often GOT or cloud is still needed.

That single change attacks systematic font substitutions and word-order scrambles with a model that fits 4 vCPU / 15 GB, and turns data you already collect into a permanent offline asset.

13. Key Links

Baberu OCR: huggingface.co/genshiai-daichi/baberu-ocr

RapidOCR: github.com/RapidAI/RapidOCR

ogkalu detectors: huggingface.co/ogkalu/comic-text-and-bubble-detector

ByT5 EN OCR correction (base): huggingface.co/yelpfeast/byt5-base-english-ocr-correction

COMICS Text+: github.com/gsoykan/comics_text_plus · arXiv 2212.14674

SCOWL / Hunspell EN: wordlist.aspell.net · github.com/en-wl/wordlist

ZentrixLabs.OcrCorrection: github.com/ZentrixLabs/ZentrixLabs.OcrCorrection

manga-ocr synthetic gen: github.com/kha-white/manga-ocr (synthetic_data_generator)

docTR-Synth-Generator: github.com/felixdittrich92/docTR-Synth-Generator

Magi reading-order: arXiv 2401.10224

Consolidated from exhaustive search across GitHub, Hugging Face, arXiv, ACL, scanlation tools, and community resources. All primary recommendations are CPU-feasible offline after first model download. Flag inflated benchmarks; measure on your own chapters.